

GALXY — Module 6: Cart

Individual Work Assignment

Deepesh — API Routes, Stale-Item Handling & QA

BACKEND

Team Lead	Dhanush
Track	Backend
Module	Module 6 — Cart (Flask · MongoDB · Frontend UI)
Primary File(s)/Area	routes/cart_routes.py, utils/cart_snapshot_helper.py
Coordinates With	Prashanth (service layer), Jaiganesh & Dhanush (API contracts/frontend), Module 8 (checkout integration)
Timeline Focus	Day 3–7: routes build. Day 5–9: stale-item logic. Day 11–13: testing lead.

Overview

Deepesh now owns every HTTP endpoint plus the graceful-degradation logic that keeps stale carts safe, and leads testing/QA for the whole module.

Tasks

- Implement **GET /api/cart** — returns items, item_count, subtotal_estimate; auto-creates cart via the service layer if missing.
- Implement **POST /api/cart/items** (Add Item) — 201 success, 400 invalid attributes, 404 missing/inactive product.
- Implement **PUT /api/cart/items/:item_id** (Update Item) — 400 invalid attributes/quantity, 404 item not found.
- Implement **DELETE /api/cart/items/:item_id** (Remove Item) and **DELETE /api/cart/clear** (Clear Cart).
- Apply `@require_auth` (Module 1) to every route — no guest cart routes in v1.
- Build `cart_snapshot_helper.py` for product_title/category_name/thumbnail snapshotting at add-time.
- Implement **read-time** stale-item flags on GET /api/cart: `is_available`: false for soft-deleted/inactive products, `needs_attention`: true (with specific error) when a silent re-validate against the current `attribute_schema` fails — never auto-modify or auto-remove the customer's selection.
- Write unit + integration tests: duplicate-item merging, stale product/attribute handling, quantity edge cases, and the clear-on-checkout contract with Module 8.

Deliverables

- routes/cart_routes.py covering all 5 endpoints with correct status codes
- utils/cart_snapshot_helper.py + stale-item flag logic
- Postman/Thunder Client collection + test suite
- QA checklist shared with the team before final merge

Watch Out / Coordination Points

- Stale-item flags must be computed fresh on every GET — never stored on the document.
- Give Jaiganesh the exact error payload shapes early so frontend error handling matches reality.
- Confirm with Module 1 how `@require_auth` injects the current user into the request context.

